

# Task 1. Prerequisites to build GraphSlam with known DA

---

## A

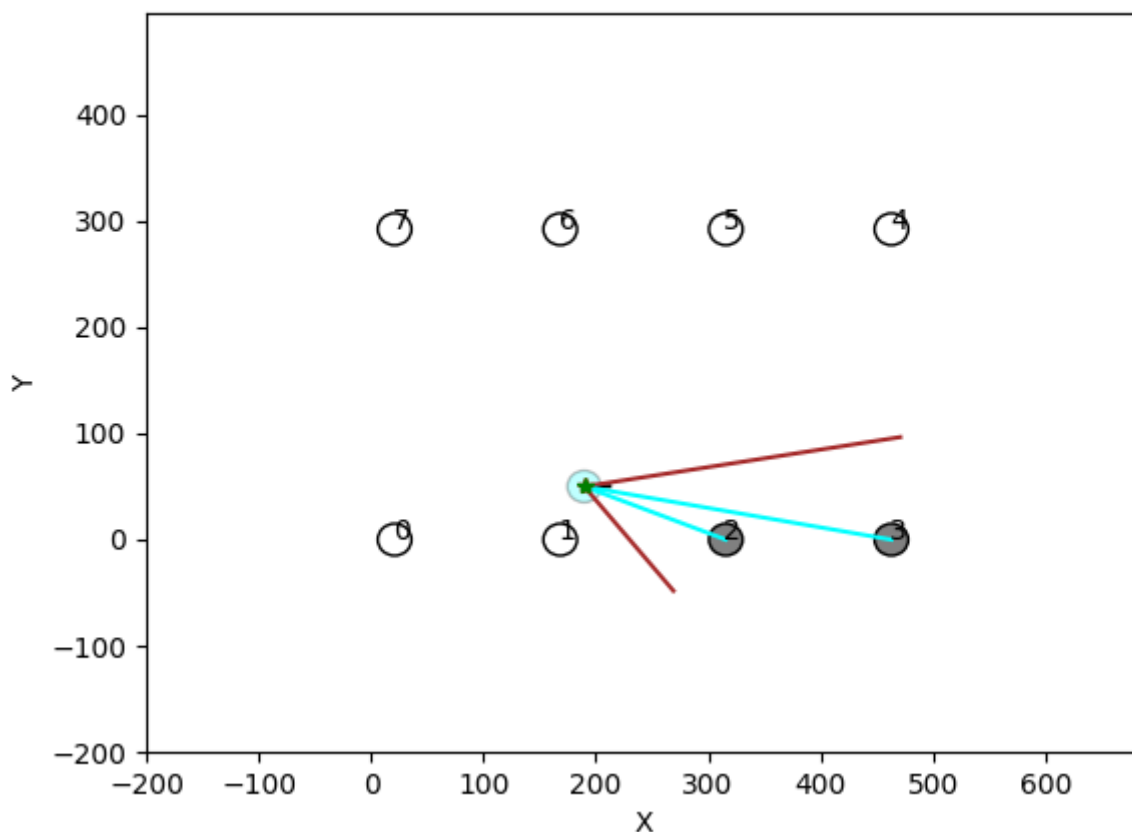
In this task the factor graph is initialized with the initial pose and other parameters.

Using `graph.print(True)` the information in the graph is printed:

```
Status of graph: Nodes = 1, Factors = 1, Eigen Factors = 0
Printing NodePose2d: 0, state =
180
 50
  0
Printing Factor: 0, obs=
180
 50
  0
Residuals=
1.94696e-308
1.94696e-308
2.2251e-308
and Information matrix
1e+12    -0    -0
    0 1e+12    -0
    0    0 1e+12
Calculated Jacobian =
0 0 0
0 0 0
0 0 0
Chi2 error = 0 and neighbour Nodes 1
```

---

The initial plot is:



## B

In this task it is required to update `GraphSlam.predict()`. The new 2d pose node is added with automatic initialization. The new factor is added based on motion, current node id, target node id and information matrix corresponding to motion.

After adding the odometry factor, the new pose is correctly initialized - `[[190, 50, 0]]`:

```
0%|
Step: 0
[array([[180.],
       [ 50.],
       [ 0.]])], array([[190.],
       [ 50.],
       [ 0.]])]
```

Status of graph: Nodes = 2, Factors = 2, Eigen Factors = 0

## C

In this task it is required to update `GraphSlam.update()`. The observations are processed in the way that new ones are initialized with nodes and for all of them factors are added based on observations, current node id, landmark node id, information matrix corresponding to observation.

After adding observation factors, landmark states are added to the graph:

```
0%|
obs ids to node id: {3: 2, 2: 3},
states: [array([[180.],
[ 50.],
[ 0.]]), array([[190.],
[ 50.],
[ 0.]]), array([[461.20052293],
[ 60.93791262]]), array([[318.71277805],
[ -3.64043779]])]
Status of graph: Nodes = 4, Factors = 4, Eigen Factors = 0
100%|
```

## D

In this task it is required to solve the optimization problem to find updates for robot and landmark poses using `solve(mrob.GN)` from `mrob` library.

The `run.py` was launched with the argument `num_steps=3`, so it is possible to see the process of updating. On the first step there are 4 nodes (current robot position, predicted robot position, 2 observations) and 4 factors. On the second step we don't see big changes in old nodes but one new node is added corresponding to the predicted robot position. On the third step we see updated nodes and extended amount of nodes and factors. So the graph is being extending and updating.

```
• (venv) (base) inna@inna-IdeaPad-Slim-5-14IRL8:~/Documents/perception-in-robotics-2025/PS3$ python3 run.py -s -f graphSlam -n 3
0%|
Step: 0
obs ids to node id: {3: 2, 2: 3},
states: [array([[180.],
[ 50.],
[ 0.]]), array([[190.],
[ 50.],
[ 0.]]), array([[459.22444851],
[ -1.43432183]]), array([[308.9016375 ],
[ 24.87542446]])]
Status of graph: Nodes = 4, Factors = 4, Eigen Factors = 0
33%|
Step: 1
obs ids to node id: {3: 2, 2: 3},
states: [array([[1.80000000e+02],
[5.00000000e+01],
[6.48345056e-42]]), array([[1.90000000e+02],
[5.00000000e+01],
[6.48345056e-42]]), array([[459.22444851],
[ -1.43432183]]), array([[308.9016375 ],
[ 24.87542446]]), array([[2.00000000e+02],
[5.00000000e+01],
[6.48345056e-42]])]
Status of graph: Nodes = 5, Factors = 7, Eigen Factors = 0
Step: 2
obs ids to node id: {3: 2, 2: 3},
states: [array([[ 1.80000000e+02],
[ 5.00000000e+01],
[-1.28914663e-27]]), array([[1.90030283e+02],
[5.00001966e+01],
[6.25067296e-05]]), array([[464.76841501],
[ 15.23179637]]), array([[302.90180797],
[ 13.25612989]]), array([[2.00030283e+02],
[5.00008217e+01],
[6.25067296e-05]]), array([[2.10030283e+02],
[5.00014468e+01],
[6.25067296e-05]])]
Status of graph: Nodes = 6, Factors = 10, Eigen Factors = 0
100%|
```

## E

In this task it is required to compare the information matrix taken from `mrob.FGraph()` and computed using adjacency matrix and a block diagonal matrix of information. The error between matrices is

computed using the l2-norm and is of several thousand units.

```

• (venv) (base) inna@inna-IdeaPad-Slim-5-14IRL8:~/Documents/perception-in-robotics-2025/PS3$ python3 run.py -s -f graphSlam -n 3
0%|
Step: 0
obs ids to node id: {3: 2, 2: 3},
states: [array([[180.],
[ 50.],
[ 0.]]), array([[190.],
[ 50.],
[ 0.]]), array([[449.8640957 ],
[ 8.27018261]]), array([[308.61037049],
[ 33.3927757 ]])]
The l2-error between computed info matrix and the one in the graph: 14282.8568570857
Status of graph: Nodes = 4, Factors = 4, Eigen Factors = 0
33%|
Step: 1
obs ids to node id: {3: 2, 2: 3},
states: [array([[1.80000000e+02],
[5.00000000e+01],
[2.46519033e-44]]), array([[1.90000000e+02],
[5.00000000e+01],
[2.46519033e-44]]), array([[449.8640957 ],
[ 8.27018261]]), array([[308.61037049],
[ 33.3927757 ]]), array([[2.00000000e+02],
[5.00000000e+01],
[2.46519033e-44]]))]
The l2-error between computed info matrix and the one in the graph: 20248.456731316594
Status of graph: Nodes = 5, Factors = 7, Eigen Factors = 0
Step: 2
obs ids to node id: {3: 2, 2: 3},
states: [array([[ 1.80000000e+02],
[ 5.00000000e+01],
[-1.10987799e-26]]), array([[1.90109076e+02],
[5.00003438e+01],
[1.43607585e-04]]), array([[440.07537625],
[-2.79436193]]), array([[311.8982896 ],
[ 31.07494598]]), array([[2.00109076e+02],
[5.00017799e+01],
[1.43607585e-04]]), array([[2.10109076e+02],
[5.00032159e+01],
[1.43607585e-04]]))]
The l2-error between computed info matrix and the one in the graph: 24819.347125795735
Status of graph: Nodes = 6, Factors = 10, Eigen Factors = 0
100%|

```

## F

In this task it is required to solve manually the normal equation and compare the solution of residuals with the one obtained using **mrob** library. The error is less than 1 unit, quite a small, so the solution is correct.

```

• (venv) (base) inna@inna-IdeaPad-Slim-5-14IRL8:~/Documents/perception-in-robotics-2025/PS3$ python3 run.py -s -f graphSlam -n 3
0%|
Step: 0
obs ids to node id: {3: 2, 2: 3},
states: [array([[180.],
[ 50.],
[ 0.]]), array([[190.],
[ 50.],
[ 0.]]), array([[449.49572582],
[ 23.1569399 ]]), array([[300.84598586],
[ 4.22315789]])]
The l2-error between normal eq solution and the mrob library solution: 3.246924647351685e-41
Status of graph: Nodes = 4, Factors = 4, Eigen Factors = 0
33%|
Step: 1
obs ids to node id: {3: 2, 2: 3},
states: [array([[ 1.80000000e+02],
[ 5.00000000e+01],
[-1.06249703e-41]]), array([[ 1.90000000e+02],
[ 5.00000000e+01],
[-1.06249703e-41]]), array([[449.49572582],
[ 23.1569399 ]]), array([[300.84598586],
[ 4.22315789]]), array([[ 2.00000000e+02],
[ 5.00000000e+01],
[-1.06249703e-41]])]
The l2-error between normal eq solution and the mrob library solution: 0.018103736720424174
Status of graph: Nodes = 5, Factors = 7, Eigen Factors = 0
Step: 2
obs ids to node id: {3: 2, 2: 3},
states: [array([[ 1.80000000e+02],
[ 5.00000000e+01],
[-1.98151013e-26]]), array([[ 1.89631076e+02],
[ 5.00011759e+01],
[-8.47371048e-05]]), array([[ 4.56069311e+02],
[-4.22831320e-01]]), array([[301.25010029],
[-12.65052157]]), array([[ 1.99631076e+02],
[ 5.00003285e+01],
[-8.47371048e-05]]), array([[ 2.09631076e+02],
[ 4.99994811e+01],
[-8.47371048e-05]])]
The l2-error between normal eq solution and the mrob library solution: 0.012618594050836549
Status of graph: Nodes = 6, Factors = 10, Eigen Factors = 0
100%|

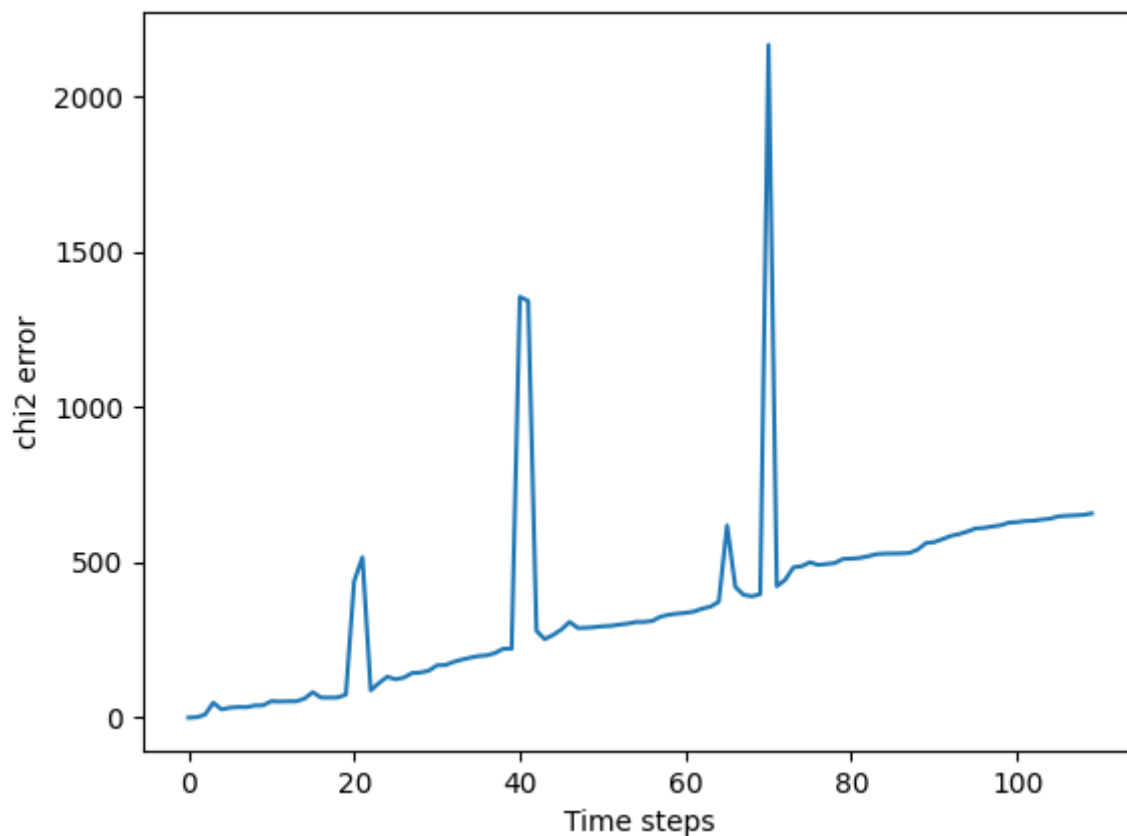
```

## Task 2: GraphSlam evaluation

---

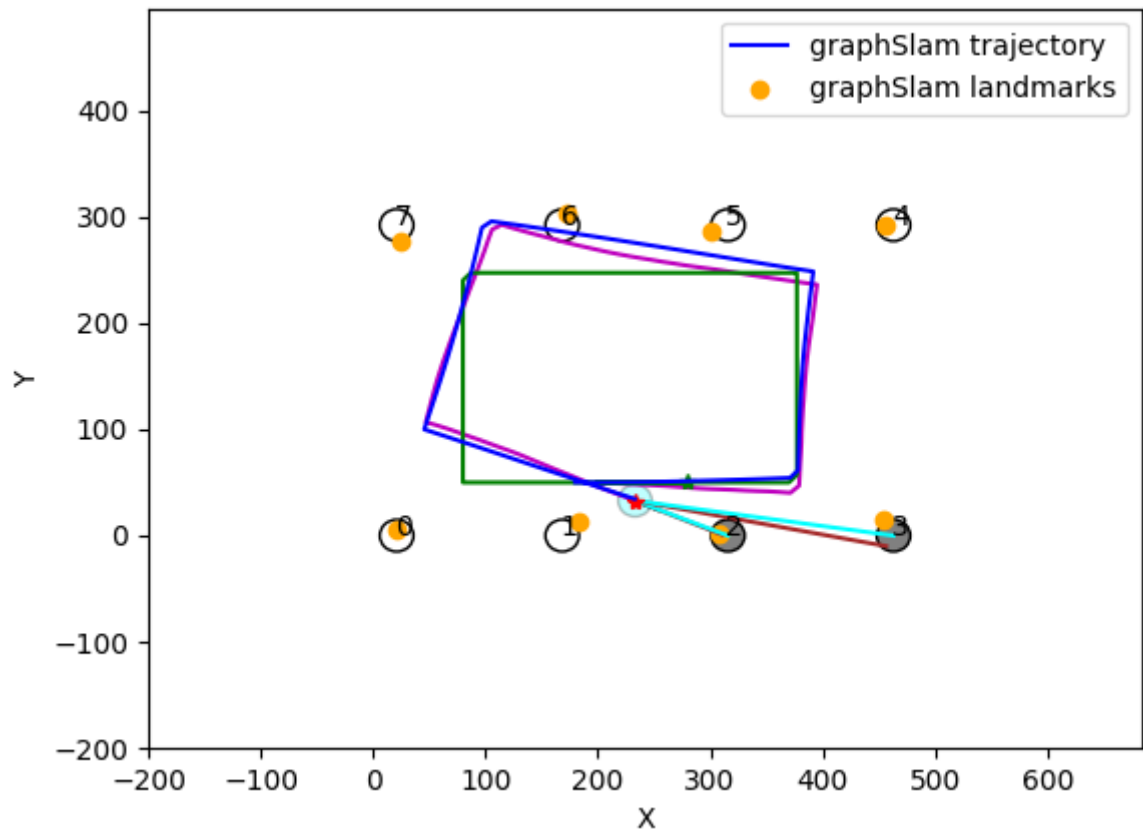
### A

In this task it is required to calculate  $\chi^2$  error on each time step. Overall, the curve is increasing within time, reaching sharp peaks approximately at turning points of robot trajectory.



### B

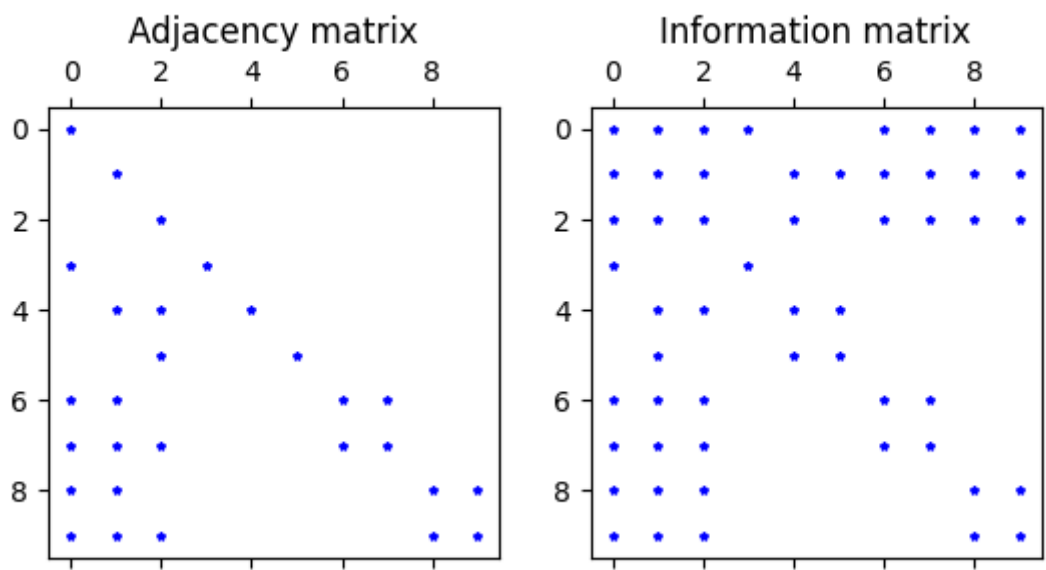
The filtered trajectory obtained from GraphSlam algorithm. In addition, there is a recorded video in the file "graphSlam.mp4".



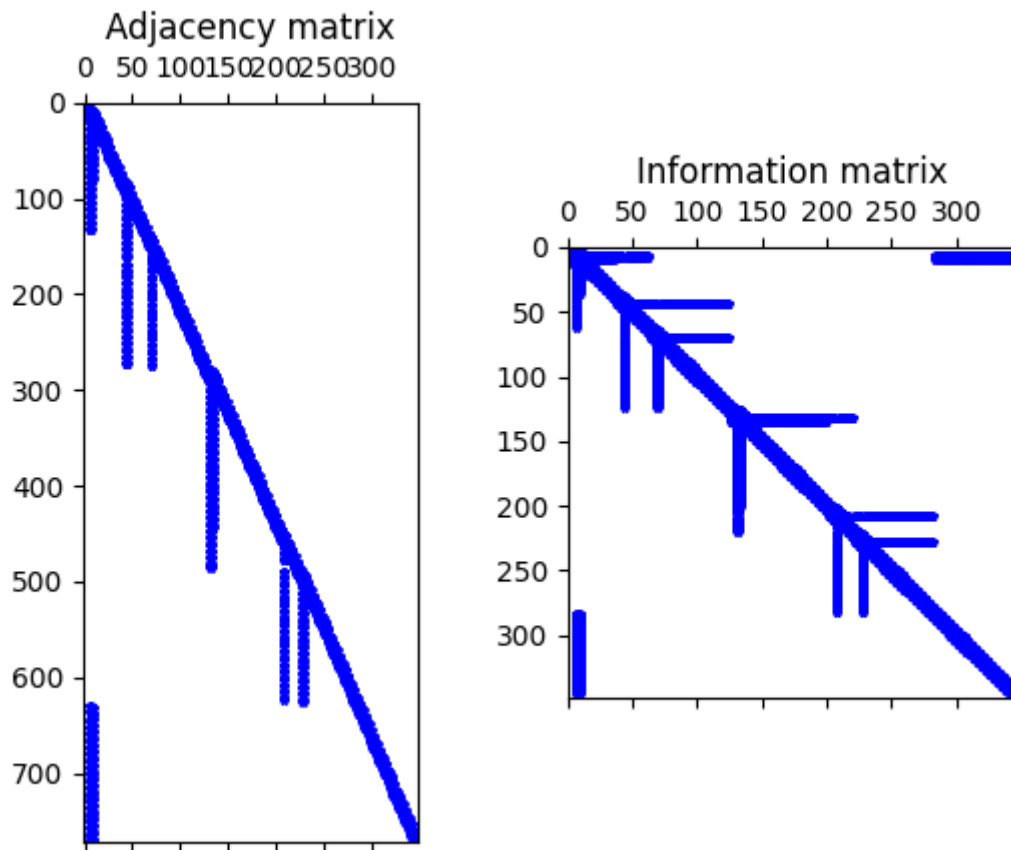
C

The adjacency and information matrix after the 1st step of iteration. Matrices are 10 by 10: 2 robot poses with 3 coordinates and 2 landmarks with 2 coordinates. We can see the connection between robot coordinates and landmarks.

The adjacency and information matrix after the 1st step of iteration (using generated data):



The adjacency and information matrix after the 100d step of iteration (using slam-evaluation-input.npy):

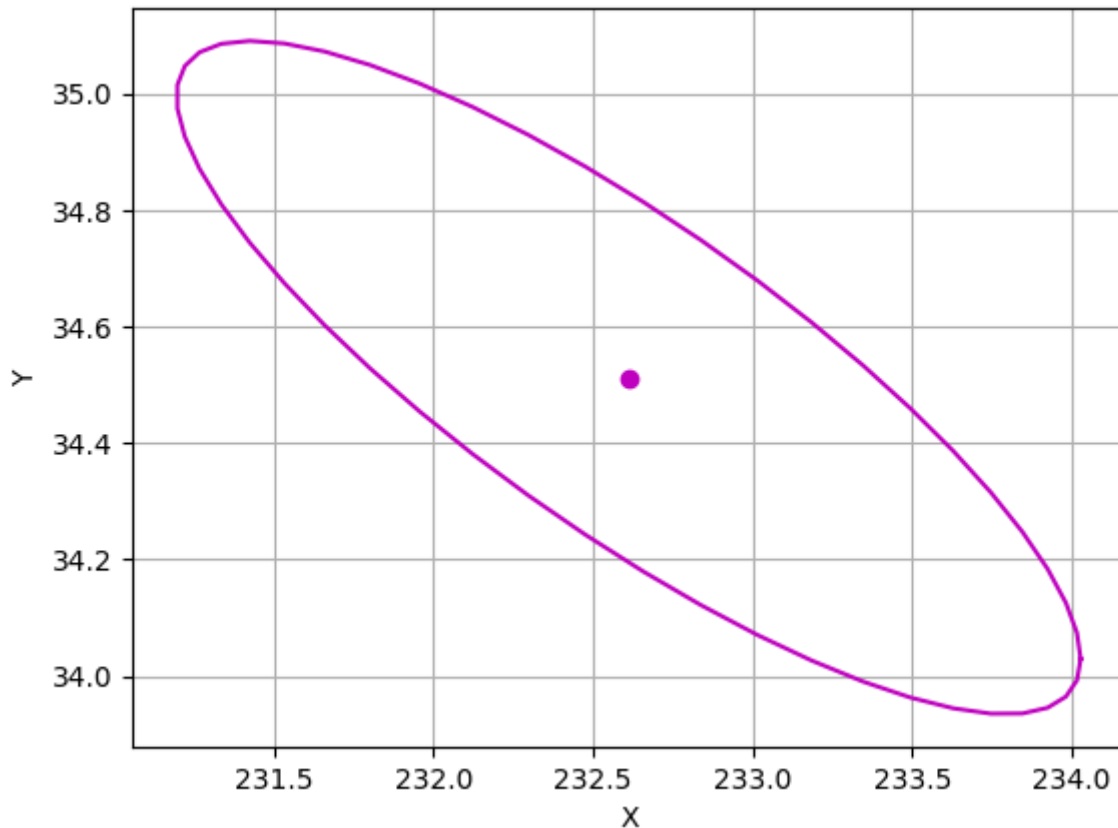


There is a common structure along both matrices. In the **adjacency matrix** the right upper part is empty because new robot states are dependent from the previous states, and a couple of robot states are linked with the same pair of landmarks (this causes "hanging tails"). The left bottom part means returning to the starting point. The **information matrix** is symmetric and shows correlations between robot positions and landmarks.

## D

In this task it is required to plot 3-sigma iso-contour for the last robot pose. For that, I extracted last pose state using `graph.get_estimated_state()` and information matrix from the graph. Using last pose state values, I found the indexes of information matrix corresponding to this pose. By inverting it, I got the

covariance. The iso-contour is plotted using function `plot2dcov()`.



## E

In this task it is required to optimize the graph in the last time step until convergence. For that, I used both Gauss-Newton (GN) and Levenberg-Marquard (LM) algorithms.

Solving using LM takes 3 iterations and the resulting  $\chi^2$  error is 893.76.

Solving using GN takes 6 iterations and the resulting  $\chi^2$  error is 898.65.

```
chi^2 error = 308880.13396533224
chi^2 error = 6323.779376933234
chi^2 error = 925.2287612280485
chi^2 error = 906.5392427565986
chi^2 error = 898.2429819909737
chi^2 error = 898.651355588647
Number of iterations: 6
```